21
GPT 학습하기

트랜스포머의 디코더만 쌓아 "다음 단어 맞히기"로 학습한 모델입니다. 한국어 GPT-2로 글을 이어 쓰게 하고, 분류 문제와 챗봇에 맞게 파인 튜닝합니다.

오늘 다루는 것

1. 지피티(GPT): 역사, 거대 언어 모델, 구조
2. GPT-2를 이용한 문장 생성
3. GPT-2를 이용한 한국어 뉴스 감성 분류
4. GPT-2를 이용한 챗봇 파인 튜닝

끝나고 나면 할 수 있는 것

1. BERT와 GPT의 차이를 구조(인코더/디코더), 보는 방향, 학습 문제 세 가지로 설명할 수 있다.
2. GPT가 글을 쓰는 과정(다음 토큰의 확률 → 하나 고르기 → 붙여서 다시 넣기)을 코드로 따라갈 수 있다.
3. GPT를 분류와 챗봇에 맞게 파인 튜닝할 때 입력을 어떻게 만드는지 말할 수 있다.

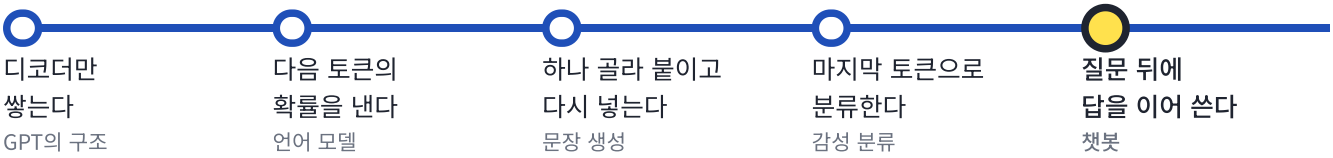
2026년 10월 16일 금요일 14:30 (10/16 자료 3/5)
한국어 GPT-2 모델 `skt/kogpt2-base-v2` (약 500MB)를 처음 실행할 때 내려받습니다. 실습 3, 4는 모델을 학습시키므로 코랩에서는 GPU 런타임을 권합니다.

21

GPT 학습하기

18장의 트랜스포머는 인코더와 디코더로 이루어져 있었습니다. 19장의 BERT는 그중 인코더만 가져갔습니다. GPT는 디코더만 가져가, 10장에서 배운 언어 모델의 일 "앞의 단어들로 다음 단어 맞히기"를 아주 크게 한 모델입니다. ChatGPT를 비롯한 지금의 거대 언어 모델이 모두 이 구조에서 나왔습니다.

왜 배우나: 오늘의 학습 경로



예시는 교재를 따라 "근육이 커지기 위해서는"이라는 글의 뒤를 이어 쓰는 것으로 시작합니다. 장 번호는 이 스터디의 자료 기준입니다(트랜스포머 18장, BERT 19장).

21-01 지피티(Generative Pre-trained Transformer, GPT)

핵심 한 줄

GPT는 트랜스포머의 디코더 층만 쌓은 언어 모델로, 앞의 토큰들만 보고 다음 토큰을 맞히도록 사전 훈련되었으며, 구조는 거의 그대로 둔 채 크기를 키워 온 것이 지금의 거대 언어 모델이다.

왜 배우나

ChatGPT를 쓰는 것과 그 안이 어떻게 생겼는지 아는 것은 다릅니다. 안을 알면 왜 한 글자씩 써 내려가는지, 왜 같은 질문에 답이 달라지는지, 왜 입력 길이에 한계가 있는지가 설명됩니다. 구조는 18장에서 이미 다 배운 것이고, 여기서는 무엇을 뺐는지만 보면 됩니다.

1) GPT의 역사와 거대 언어 모델

정의

- GPT(Generative Pre-trained Transformer): 글을 생성하는(Generative), 사전 훈련된(Pre-trained), 트랜스포머(Transformer). OpenAI가 2018년부터 내놓은 모델의 이름입니다.
- 언어 모델(language model)(10장): 앞의 단어들이 주어졌을 때 다음 단어의 확률을 내는 모델.
- 거대 언어 모델(Large Language Model, LLM): 딥 러닝 언어 모델 중 파라미터 수가 충분히 큰 것. GPT는 수많은 언어 모델 중 하나이고, Gemini, Claude, CLOVA X도 언어 모델입니다.

	GPT-1 (2018)	GPT-2 (2019)	GPT-3 (2020)	이후
파라미터	1억 1,700만 개	15억 개	1,750억 개	ChatGPT(GPT-3.5, 2022), GPT-4(2023), GPT-4o(2024). 크기는 공식적으로 공개되지 않았다
디코더 층	12	48	96	
한 번에 읽는 토큰 수	512	1,024	2,048	
벡터의 크기	768	1,600	12,288	

GPT-1에서 GPT-3까지 구조는 거의 바뀌지 않았습니다. 바뀐 것은 크기와 학습 데이터의 양입니다. 모델이 충분히 커지자, 파인 튜닝 없이 사전 훈련만으로도 지시를 따르는 것처럼 보이는 능력이 나타났습니다. 그것이 거대 언어 모델의 출발점입니다.

예시 적용

이 장의 실습에서 쓰는 `skt/kogpt2-base-v2` 는 한국어 글로 사전 훈련한 GPT-2의 가장 작은 판입니다(실습 1의 출력).

	KoGPT-2 (실습)	BERT-Base (19장)

층	디코더 12층	인코더 12층
벡터의 크기 / 헤드	768 / 12	768 / 12
단어 집합	51,200	30,522
한 번에 읽는 토큰 수	1,024	512
파라미터	125,164,032개	109,482,240개

크기는 BERT-Base와 거의 같습니다. 위 표의 GPT-3는 이것의 약 1,400배입니다.

2) GPT의 아키텍처

정의

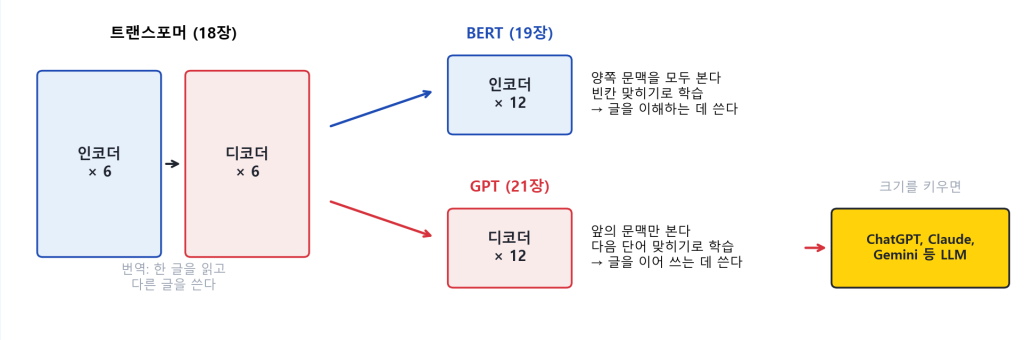


그림 1 트랜스포머에서 갈라진 두 갈래. BERT는 인코더만, GPT는 디코더만 쌓았다

- 디코더 전용(decoder-only) 구조: 18장의 디코더 층에서 인코더-디코더 어텐션을 뺀 것. 읽어 올 인코더가 없기 때문입니다. 남는 것은 마스크드 셀프 어텐션과 피드 포워드 두 개입니다.
- 마스크드 셀프 어텐션(18장): 각 토큰이 자기와 자기 앞의 토큰만 참고하도록, 뒤쪽 토큰의 어텐션 점수를 $-\infty$로 가린 셀프 어텐션. 룩어헤드 마스크를 씌운 것입니다.
- 층 정규화의 위치: GPT-1은 18장처럼 서브층 뒤에, GPT-2부터는 서브층 앞에 둡니다. 깊게 쌓아도 학습이 안정적이어서입니다.

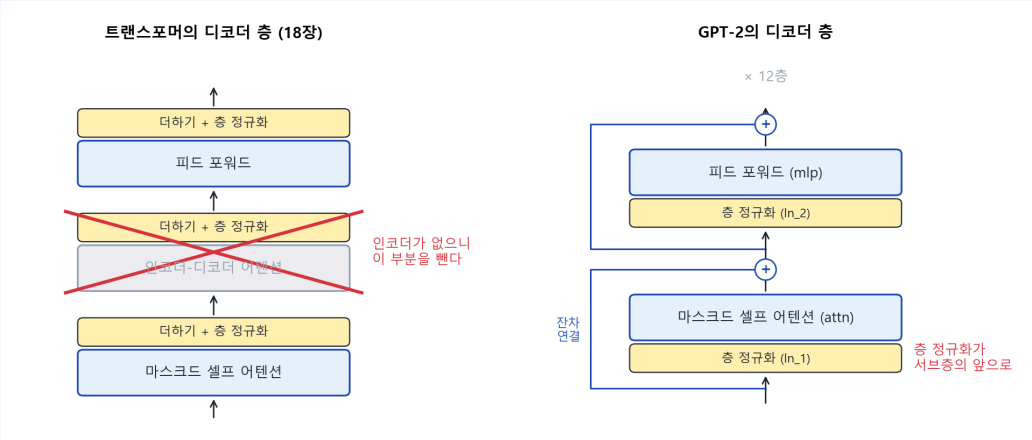


그림 2 18장의 디코더 층(왼쪽)과 GPT-2의 디코더 층(오른쪽). 괄호 안은 실습 1의 출력에 나오는 부품 이름

입력과 출력의 모양. 토큰 n개가 들어가면 자리마다 "그다음 토큰"의 점수가 단어 집합 크기 $|V|$만큼 나옵니다.

$$(n,) \xrightarrow{\text{임베딩}+\text{위치}} (n,\ D) \xrightarrow{\text{디코더 층}\times L} (n,\ D) \xrightarrow{\text{선형층}} (n,\ |V|)$$

$$P(w_{t+1} \mid w_1, \ldots, w_t) = \mathrm{softmax}(W\, h_t)_{w_{t+1}}$$

기호	뜻	KoGPT-2에서
h_t	t번째 자리의 마지막 층 출력. w_1부터 w_t까지만 반영한다	크기 768
W	벡터를 단어 집합 크기의 점수로 바꾸는 선형층	768 → 51,200
$L,\ D,\ \|V\|$	층 수, 벡터의 크기, 단어 집합의 크기	12, 768, 51,200

	BERT	GPT
가져온 부분	인코더	디코더
보는 방향	앞뒤 모두 (양방향)	앞만 (단방향)
사전 훈련 문제	가린 단어 맞히기, 다음 문장 맞히기	다음 단어 맞히기
잘하는 일	분류, 태깅, 검색 등 글을 이해하는 일	글을 이어 쓰는 일

예시 적용

"오늘 날씨가 정말"을 넣으면 토큰은 `▁오늘`, `▁날씨가`, `▁정말` 세 개이고, 모양은 (1, 3) → (1, 3, 51200)입니다. 자리마다 51,200개 토큰 각각에 대한 점수가 나옵니다(실습 1).

여기까지 읽고	가장 점수가 높은 다음 토큰
오늘	,
오늘 날씨가	추
오늘 날씨가 정말	좋

정말 뒤를 보지 못하는가. 마지막 토큰 "정말"을 "매우"로 바꿔 다시 넣었더니, 앞의 두 자리의 출력은 소수점 넷째 자리까지 그대로였고 마지막 자리의 출력만 달라졌습니다. 각 자리의 출력이 그 자리와 그 앞의 토큰에만 달려 있다는 뜻입니다. BERT였다면 한 토큰만 바꿔도 모든 자리의 출력이 달라집니다.

비유

읽는 사람과 쓰는 사람

BERT는 완성된 글을 앞뒤로 훑어보며 빈칸을 채우는 독자입니다. GPT는 지금까지 쓴 글만 보고 다음 낱말을 정하는 작가입니다. 작가는 아직 쓰지 않은 뒷부분을 볼 수 없습니다.

빈칸을 채우는 독자 = = BERT (양방향, MLM)
다음 낱말을 정하는 작가 = = GPT (단방향, 다음 토큰 맞히기)
아직 쓰지 않은 뒷부분 = = 마스크로 가린 뒤쪽 토큰

실습 1 KoGPT-2의 크기, 입력과 출력의 모양, 뒤를 보지 않는다는 것 확인하기

```
# (처음 한 번 한국어 GPT-2 모델 500MB 다운로드)
import torch
from transformers import GPT2LMHeadModel, PreTrainedTokenizerFast

# 이 모델의 토크나이저는 AutoTokenizer 대신 아래 클래스로 불러와야 한다
tokenizer = PreTrainedTokenizerFast.from_pretrained(
    "skt/kogpt2-base-v2", bos_token="</s>", eos_token="</s>", unk_token="<unk>",
    pad_token="<pad>", mask_token="<mask>")
model = GPT2LMHeadModel.from_pretrained("skt/kogpt2-base-v2")
model.eval()

# 1) 크기: 트랜스포머 디코더 층 12개
config = model.config
total = sum(p.numel() for p in model.parameters())
print(f"층 {config.n_layer}개, 은닉 크기 {config.n_embd}, 헤드 {config.n_head}개,",
      f"단어 집합 {config.vocab_size}, 최대 길이 {config.n_positions}")
print(f"파라미터: {total:,}개")
block = model.transformer.h[0]
print("디코더 층 하나의 부품:", [name for name, _ in block.named_children()])

# 2) 입력과 출력: 토큰마다 "다음 토큰"의 점수가 단어 집합 크기만큼 나온다
text = "오늘 날씨가 정말"
input_ids = tokenizer.encode(text, return_tensors="pt")
print("\n토큰:", tokenizer.convert_ids_to_tokens(input_ids[0]))
with torch.no_grad():
    logits = model(input_ids).logits
print("입력:", tuple(input_ids.shape), "→ 점수:", tuple(logits.shape))
for position, token in enumerate(tokenizer.convert_ids_to_tokens(input_ids[0])):
    best = tokenizer.decode(logits[0, position].argmax().item())
    print(f"  '{token}'까지 읽고 예측한 다음 토큰: '{best}'")

# 3) 뒤의 단어를 보지 못하는가: 마지막 토큰을 바꿔도 앞 자리의 출력은 그대로다
other = input_ids.clone()
other[0, -1] = tokenizer.encode("매우")[0]        # '정말' → '매우'
with torch.no_grad():
    logits2 = model(other).logits
same = torch.allclose(logits[0, :-1], logits2[0, :-1], atol=1e-4)
print("\n마지막 토큰을 바꿨을 때 앞 자리들의 출력이 그대로인가?", same)
print("마지막 자리의 출력은 그대로인가?",
      torch.allclose(logits[0, -1], logits2[0, -1], atol=1e-4))
```

실행 결과

```
층 12개, 은닉 크기 768, 헤드 12개, 단어 집합 51200, 최대 길이 1024
파라미터: 125,164,032개
디코더 층 하나의 부품: ['ln_1', 'attn', 'ln_2', 'mlp']

토큰: ['▁오늘', '▁날씨가', '▁정말']
입력: (1, 3) → 점수: (1, 3, 51200)
  '▁오늘'까지 읽고 예측한 다음 토큰: ','
  '▁날씨가'까지 읽고 예측한 다음 토큰: '추'
  '▁정말'까지 읽고 예측한 다음 토큰: '좋'

마지막 토큰을 바꿨을 때 앞 자리들의 출력이 그대로인가? True
마지막 자리의 출력은 그대로인가? False
```

직접 해 보기 `text`를 "나는 오늘 학교에"로 바꿔 자리마다 예측한 다음 토큰을 보세요. 3)에서 마지막 토큰이 아니라 **첫 토큰**을 바꾸면 어느 자리의 출력이 달라질까요?

21-01 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 GPT는 트랜스포머의 인코더를 쌓은 모델이다. (O / X) 이름 _____

2 GPT-1에서 GPT-3로 가면서 구조가 완전히 새로 설계되었다. (O / X) 이름 _____

3 GPT의 디코더 층에 없는 것은? 이름 _____

① 마스크드 셀프 어텐션 ② 인코더-디코더 어텐션 ③ 피드 포워드

4 GPT의 사전 훈련 문제는? 이름 _____

① 가린 단어 맞히기 ② 다음 문장 맞히기 ③ 다음 단어 맞히기

5 파라미터 수가 충분히 큰 언어 모델을 _____(LLM)이라고 한다. 이름 _____

6 KoGPT-2에 토큰 3개를 넣으면 출력의 모양은 (1, 3, _____)이다. 이름 _____

7 실습에서 마지막 토큰을 바꿔도 앞 자리의 출력이 그대로였던 이유를 설명해 보세요. 이름 _____

원문: 21-01 지피티(Generative Pre-trained Transformer, GPT) — https://wikidocs.net/256163 (교재 본문에는 GPT-1의 공개 연도가 2019년과 2018년으로 섞여 있어 2018년으로 통일했습니다. 교재가 가리키는 "16장 트랜스포머, 17장 BERT"는 이 스터디 자료의 18장, 19장입니다. 교재에는 실습이 없어 모델의 크기와 단방향성을 확인하는 실습을 추가했습니다)

21-02 GPT-2를 이용한 문장 생성

핵심 한 줄

GPT는 다음 토큰의 확률만 낸다. 그 확률에서 토큰 하나를 고르고, 입력 뒤에 붙여 다시 넣는 일을 반복하는 것이 문장 생성이고, 어떻게 고르느냐에 따라 글이 달라진다.

왜 배우나

ChatGPT가 글자를 하나씩 써 내려가는 것처럼 보이는 것은 연출이 아니라 실제 동작입니다. 같은 질문에 답이 매번 다른 것도 이 절의 "고르는 방법" 때문입니다. 16장의 seq2seq 디코더가 한 단어씩 쓰던 것과 같은 방식입니다.

1) 자기 회귀 생성과 디코딩 방법

정의

- 자기 회귀(autoregressive) 생성: 모델이 낸 출력을 다음 단계의 입력에 붙여 다시 넣는 방식의 생성.
- 디코딩(decoding) 방법: 다음 토큰의 확률 분포에서 실제로 토큰 하나를 고르는 규칙.

$$p_t = \mathrm{softmax}(z_t), \qquad w_{t+1} \sim \text{고르기}(p_t), \qquad \text{입력} \leftarrow (w_1, \ldots, w_t,\ w_{t+1})$$

기호	뜻
z_t	마지막 자리의 점수(로짓). 크기는 단어 집합의 크기
p_t	다음 토큰의 확률 분포

고르는 방법	규칙	특징
그리디 (greedy)	확률이 가장 높은 토큰 하나	실행할 때마다 같은 글. 같은 말을 되풀이하기 쉽다
top-k 샘플링	상위 k개 중에서 뽑는다	실행할 때마다 다른 글. k가 크면 엉뚱해진다
반복 벌점 (repetition penalty)	이미 쓴 토큰의 점수를 깎는다	되풀이를 줄인다

허깅페이스에서는 `model.generate(input_ids, ...)` 가 이 반복을 대신 해 줍니다. `do_sample=True, top_k=5` , `repetition_penalty=2.0` 같은 인자로 고르는 방법을 정합니다.

예시 적용

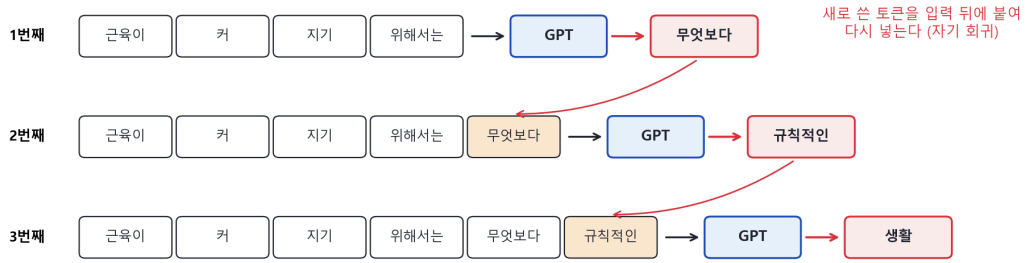


그림 3 "근육이 커지기 위해서는"의 뒤를 한 토큰씩 이어 쓰는 과정

첫 단계. "근육이 커지기 위해서는"은 토큰 4개 `▁근육이`, `▁커`, `지기`, `▁위해서는`(번호 33245, 10114, 12748, 11357)입니다. 마지막 자리의 확률에서 상위 5개는 다음과 같습니다(실습 2).

다음 토큰 후보	무엇보다	우선	반드시	피부	무엇보다도
확률	0.020	0.012	0.009	0.009	0.008

1등의 확률이 0.02뿐입니다. 51,200개 중에서 올 수 있는 말이 그만큼 많다는 뜻이고, 19-03에서 "[MASK]에 출근을 했다"의 1등이 0.08이었던 것과 같은 상황입니다.

고르는 방법에 따른 차이.

방법	이어 쓴 글
그리디	…무엇보다 규칙적인 생활습관이 중요하다. 특히, 아침식사는 단백질과 비타민, 무기질 등 영양소가 풍부한 음식을 골고루 섭취하는
상위 5개에서 무작위 (시드 0)	…피부 타입에 맞게 골라주는 것만이 도움이 된다고 말한다. 이렇게 하면 피부의 탄력도 높아져…
상위 5개에서 무작위 (시드 1)	…우선 균형 잡힌 식사를 해야 하며 식사량이 부족할 때는 충분한 수면이 중요하다.…

시드 0에서는 첫 토큰으로 4등인 "피부"가 뽑혔고, 그 뒤로 글 전체가 피부 이야기로 흘러갔습니다. 근육 이야기로 돌아오지 못합니다. 한 번 고른 토큰이 그 뒤의 모든 확률을 바꾸기 때문입니다. 무작위로 뽑으면 글이 다양해지는 대신 이런 일이 생깁니다.

문법은 모두 자연스럽지만 내용이 사실인지는 보장되지 않습니다("하루 30분 이상 충분한 수면"). 모델은 그럴듯한 다음 토큰을 고를 뿐입니다.

비유

휴대폰 자판의 추천 단어만 눌러 문장 쓰기

자판 위에 뜨는 추천 단어 중 맨 왼쪽만 계속 누르면 늘 같은 문장이 나옵니다. 셋 중 아무거나 누르면 매번 다른 문장이 나오고, 가끔 엉뚱한 데로 흘러갑니다.

추천 단어 목록	= = 다음 토큰의 확률 분포
맨 왼쪽만 누르기	= = 그리디
셋 중 아무거나 누르기	= = top-k 샘플링 (k = 3)
누른 단어가 문장에 붙는 것	= = 자기 회귀

실습 2

KoGPT-2로 글 이어 쓰기: 그리디, generate, 상위 5개 샘플링

```
import random

import torch
from transformers import GPT2LMHeadModel, PreTrainedTokenizerFast

# 이 모델의 토크나이저는 AutoTokenizer 대신 아래 클래스로 불러와야 한다
tokenizer = PreTrainedTokenizerFast.from_pretrained(
    "skt/kogpt2-base-v2", bos_token="</s>", eos_token="</s>", unk_token="<unk>",
    pad_token="<pad>", mask_token="<mask>")
model = GPT2LMHeadModel.from_pretrained("skt/kogpt2-base-v2")
model.eval()

sent = "근육이 커지기 위해서는"
input_ids = tokenizer.encode(sent, return_tensors="pt")
print("입력 토큰:", tokenizer.convert_ids_to_tokens(input_ids[0]),
      input_ids.tolist())

# 1) 다음 토큰의 후보 5개와 확률
with torch.no_grad():
    logits = model(input_ids).logits[0, -1]          # 마지막 자리의 점수 (51200,)
probs = torch.softmax(logits, dim=0)
top5 = probs.topk(5)
print("다음 토큰 후보:", [(tokenizer.decode(i), round(p.item(), 3))
                    for p, i in zip(top5.values, top5.indices)])

# 2) 직접 이어 쓰기: 가장 확률이 높은 토큰을 붙이는 일을 반복한다 (그리디)
ids = input_ids.clone()
for _ in range(25):
    with torch.no_grad():
        next_id = model(ids).logits[0, -1].argmax()
    ids = torch.cat([ids, next_id.view(1, 1)], dim=1)
print("\n[가장 높은 것만 고르기]\n", tokenizer.decode(ids[0]))

# 3) generate 함수: 같은 일을 해 준다. 반복 벌점을 주면 같은 말을 덜 되풀이한다
out = model.generate(input_ids, max_length=60, repetition_penalty=2.0,
                     pad_token_id=tokenizer.eos_token_id)
print("\n[generate, 반복 벌점 2.0]\n", tokenizer.decode(out[0]))

# 4) 상위 5개 중에서 무작위로 고르기: 실행할 때마다 다른 글이 나온다
for seed in (0, 1):
    random.seed(seed)
    ids = input_ids.clone()
    for _ in range(30):
        with torch.no_grad():
            logits = model(ids).logits[0, -1]
        token_id = random.choice(logits.topk(5).indices.tolist())
        ids = torch.cat([ids, torch.tensor([[token_id]])], dim=1)
    print(f"\n[상위 5개에서 무작위, 시드 {seed}]\n", tokenizer.decode(ids[0]))
```

실행 결과

```
입력 토큰: ['▁근육이', '▁커', '지기', '▁위해서는'] [[33245, 10114, 12748, 11357]]
다음 토큰 후보: [('무엇보다', 0.02), ('우선', 0.012), ('반드시', 0.009), ('피부', 0.009),
('무엇보다도', 0.008)]

[가장 높은 것만 고르기]
 근육이 커지기 위해서는 무엇보다 규칙적인 생활습관이 중요하다.
특히, 아침식사는 단백질과 비타민, 무기질 등 영양소가 풍부한 음식을 골고루 섭취하는

[generate, 반복 벌점 2.0]
 근육이 커지기 위해서는 무엇보다 규칙적인 생활습관이 중요하다.
특히, 아침식사는 단백질과 비타민이 풍부한 과일과 채소를 많이 섭취하는 것이 좋다.
또한 하루 30분 이상 충분한 수면을 취하는 것도 도움이 된다.
아침 식사를 거르지 않고 규칙적으로 운동을 하면 혈액순환에 도움을 줄 뿐만 아니라 신진대사를 촉진

[상위 5개에서 무작위, 시드 0]
 근육이 커지기 위해서는 피부 타입에 맞게 골라주는 것만이 도움이 된다고 말한다.
이렇게 하면 피부의 탄력도 높아져 주름이나 칙임 등의 피부노화가 개선되고, 피부탄

[상위 5개에서 무작위, 시드 1]
 근육이 커지기 위해서는 우선 균형 잡힌 식사를 해야 하며 식사량이 부족할 때는 충분한 수면이
중요하다.
아침 식사량을 줄이고 규칙적인 운동을 하면 체중을 감량을 할 수도 있고
```

직접 해 보기 `sent`를 자기 문장으로 바꿔 보세요. 4)에서 상위 5개를 상위 50개로 바꾸면 글이 어떻게 달라지나요? 3)에서 `repetition_penalty`를 빼면 어떤 일이 생기나요?

확인 문제

21-02 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 GPT는 한 번의 계산으로 문장 전체를 한꺼번에 출력한다. (O / X) 이름 _____

2 그리디 방법은 실행할 때마다 다른 글을 만든다. (O / X) 이름 _____

3 다음 토큰을 고를 때 쓰는 것은? 이름 _____

① 첫 자리의 점수 ② 마지막 자리의 점수 ③ 모든 자리 점수의 평균

4 이미 쓴 토큰의 점수를 깎아 되풀이를 줄이는 것은? 이름 _____

① 반복 벌점 ② 어텐션 마스크 ③ 드롭아웃

5 모델의 출력을 다음 입력에 붙여 다시 넣는 생성 방식을 _____ 생성이라고 한다. 이름 _____

6 "근육이 커지기 위해서는" 다음 토큰 1등 "무엇보다"의 확률은 _____였다. 이름 _____

7 시드 0의 글이 피부 이야기로 흘러가 돌아오지 못한 이유를 설명해 보세요. 이름 _____

원문: 21-02 GPT-2를 이용한 문장 생성 — https://wikidocs.net/217618 (교재는 토크나이저를 AutoTokenizer로 불러옵니다. 지금의 transformers 버전에서는 그렇게 하면 한글이 바이트 조각으로 쪼개져, 이 모델에 맞는 PreTrainedTokenizerFast로 불러왔습니다. 토큰 번호 33245, 10114, 12748, 11357은 교재와 같습니다. 그리디로 직접 이어 쓰는 부분과 다음 토큰의 확률을 보는 부분은 추가한 것입니다)

21-03 GPT-2를 이용한 한국어 뉴스 감성 분류

핵심 한 줄

GPT로 문장을 분류할 때는 문장 전체를 읽은 유일한 자리인 마지막 토큰의 벡터에 선형층을 붙이고, 나머지는 BERT의 파인 튜닝과 똑같이 학습시킨다.

왜 배우나

20-01에서 BERT로 풀었던 금융 뉴스 분류를 GPT로 다시 풉니다. 같은 문제를 구조가 다른 모델로 풀어 보면, "앞만 본다"는 GPT의 성질이 파인 튜닝의 모양을 어떻게 바꾸는지 보입니다.

1) 디코더 모델로 분류하기

정의

- 마지막 토큰의 벡터: GPT의 각 자리의 출력은 그 자리까지의 토큰만 반영합니다(21-01). 문장 전체를 본 것은 마지막 토큰의 자리뿐이므로, 분류에는 그 벡터를 씁니다. BERT의 [CLS]가 하던 일을 마지막 토큰이 합니다.
- 패딩 토큰: 배치 안에서 문장의 길이를 맞추려고 뒤에 채우는 토큰. "마지막 실제 토큰"이 어디인지 찾으려면 모델이 패딩 토큰의 번호를 알아야 합니다 (`model.config.pad_token_id`).

$$(n,) \xrightarrow{\text{GPT}} (n,\ D) \xrightarrow{\text{마지막 실제 토큰만 꺼냄}} (D,) \xrightarrow{\text{선형층}} (C,)$$

$$\hat{y} = \text{softmax}(W\, h_n), \qquad \text{손실} = -\log \hat{y}_{\text{정답}}$$

기호	뜻	이 실습에서
h_n	마지막 실제 토큰 자리의 출력 벡터	크기 768
W	새로 붙인 선형층 (편향 없음)	768 → 3
C	클래스의 수	3 (중립, 긍정, 부정)

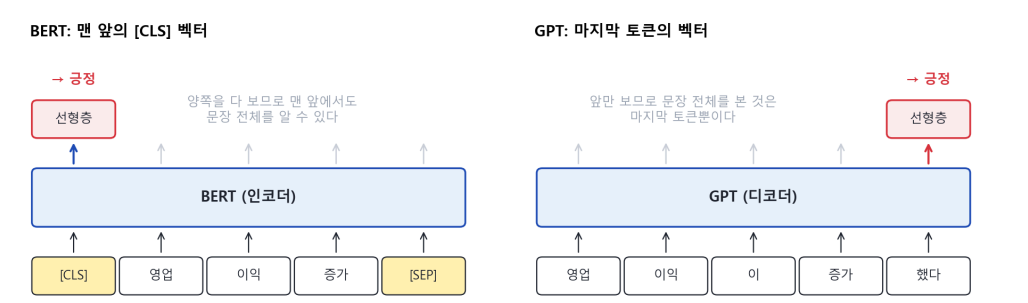


그림 4 분류에 쓰는 벡터의 자리. BERT는 맨 앞, GPT는 맨 뒤

허깅페이스에서는 20-01과 같은 `AutoModelForSequenceClassification` 을 그대로 씁니다. 모델 이름이 GPT-2이면 위와 같이 마지막 토큰을 쓰는 분류 모델을 만들어 줍니다.

예시 적용

20-01과 같은 금융 뉴스 문장을 이번에는 중립까지 셋으로 나눴습니다. 학습 1,200문장, 테스트 300문장, 1 에포크입니다(실습 3).

	테스트 정확도
모두 "중립"이라고 찍기 (기준선)	0.5933
파인 튜닝 전 (출력층이 무작위)	0.2267
1 에포크 뒤	0.8033

새 문장	중립	긍정	부정
3분기 영업이익이 전년 대비 20% 증가했다.	0.00	**0.98**	0.02
회사는 실적 부진으로 직원 200명을 감원할 계획이다.	0.03	0.04	**0.92**
회사의 본사는 헬싱키에 있다.	**0.94**	0.05	0.01

한 번 훑은 것만으로 기준선보다 0.21 높습니다. 출력층은 `Linear(768 → 3, bias=False)` 로, 위에서 정의한 대로 편향이 없습니다. 20-01(긍정/부정 둘, 0.92)과 숫자를 바로 비교할 수는 없습니다. 클래스 수와 데이터가 다르기 때문입니다.

실습 3

KoGPT-2로 금융 뉴스 문장을 중립/긍정/부정으로 분류하기

```
# GPT-2로 금융 뉴스 문장을 중립/긍정/부정으로 분류한다 (CPU로 약 15분)
import urllib.request

import pandas as pd
import torch
from sklearn.model_selection import train_test_split
from torch.utils.data import DataLoader, TensorDataset
from transformers import AutoModelForSequenceClassification, PreTrainedTokenizerFast

MODEL = "skt/kogpt2-base-v2"
N_TRAIN, N_TEST, EPOCHS, MAX_LEN = 1200, 300, 1, 64      # 교재는 전체 데이터, GPU
NAMES = ["neutral", "positive", "negative"]             # 0, 1, 2
torch.manual_seed(0)

# 1) 데이터: 20-01과 같은 금융 뉴스 문장. 이번에는 중립까지 세 가지를 모두 쓴다
url = ("https://raw.githubusercontent.com/ukairia777/finance_sentiment_corpus/"
       "main/finance_data.csv")
urllib.request.urlretrieve(url, "finance_data.csv")
df = pd.read_csv("finance_data.csv")
texts = df["kor_sentence"].tolist()
labels = [NAMES.index(x) for x in df["labels"]]
X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, train_size=N_TRAIN, test_size=N_TEST, random_state=0,
    stratify=labels)
print("학습:", len(X_train), " 테스트:", len(X_test))

# 2) 토크나이저: GPT-2에는 [CLS]가 없다. 패딩 토큰을 알려 줘야 한다
tokenizer = PreTrainedTokenizerFast.from_pretrained(
    MODEL, bos_token="</s>", eos_token="</s>", unk_token="<unk>", pad_token="<pad>",
    mask_token="<mask>")
enc = tokenizer(X_train[0])
print("토큰:", tokenizer.convert_ids_to_tokens(enc["input_ids"])[:10], "...")

def encode(texts, labels):
    enc = tokenizer(texts, padding=True, truncation=True, max_length=MAX_LEN,
                    return_tensors="pt")
    return TensorDataset(enc["input_ids"], enc["attention_mask"],
                         torch.tensor(labels))

train_set, test_set = encode(X_train, y_train), encode(X_test, y_test)

# 3) 모델: GPT-2 본체 + 마지막 토큰의 벡터를 받는 선형층(출력 3개)
model = AutoModelForSequenceClassification.from_pretrained(MODEL, num_labels=3)
```

```
# 패딩 토큰의 번호: 어디가 마지막 실제 토큰인지 찾는 데 쓴다
model.config.pad_token_id = tokenizer.pad_token_id
print("출력층:", model.score)
optimizer = torch.optim.AdamW(model.parameters(), lr=2e-5)

def accuracy(dataset):
    model.eval()
    correct = 0
    with torch.no_grad():
        for ids, mask, y in DataLoader(dataset, batch_size=32):
            logits = model(input_ids=ids, attention_mask=mask).logits
            correct += (logits.argmax(dim=1) == y).sum().item()
    return correct / len(dataset)

print(f"파인 튜닝 전 테스트 정확도: {accuracy(test_set):.4f}")
for epoch in range(1, EPOCHS + 1):
    model.train()
    total = 0.0
    for ids, mask, y in DataLoader(train_set, batch_size=16, shuffle=True):
        loss = model(input_ids=ids, attention_mask=mask, labels=y).loss
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total += loss.item() * len(y)
    print(f"에포크 {epoch}: 학습 손실 {total / len(train_set):.4f},",
          f"테스트 정확도 {accuracy(test_set):.4f}")
majority = max(y_test.count(k) for k in range(3)) / len(y_test)
print(f"모두 '중립'이라고 찍을 때의 정확도: {majority:.4f}")

model.eval()
for text in ["3분기 영업이익이 전년 대비 20% 증가했다.",
             "회사는 실적 부진으로 직원 200명을 감원할 계획이다.",
             "회사의 본사는 헬싱키에 있다."]:
    enc = tokenizer(text, return_tensors="pt")
    with torch.no_grad():
        probs = torch.softmax(model(**enc).logits, dim=1)[0]
    names = ["중립", "긍정", "부정"]
    result = ", ".join(f"{k} {p:.2f}" for k, p in zip(names, probs))
    print(f"  {text} → {result}")
```

실행 결과

```
학습: 1200  테스트: 300
토큰: ['▁2010년', '▁4~6', '월', '▁순', '매출', '액은', '▁전', '년', '▁동', '월'] ...
출력층: Linear(in_features=768, out_features=3, bias=False)
파인 튜닝 전 테스트 정확도: 0.2267
에포크 1: 학습 손실 0.6675, 테스트 정확도 0.8033
모두 '중립'이라고 찍을 때의 정확도: 0.5933
  3분기 영업이익이 전년 대비 20% 증가했다. → 중립 0.00, 긍정 0.98, 부정 0.02
  회사는 실적 부진으로 직원 200명을 감원할 계획이다. → 중립 0.03, 긍정 0.04, 부정 0.92
  회사의 본사는 헬싱키에 있다. → 중립 0.94, 긍정 0.05, 부정 0.01
```

직접 해 보기 20-01의 코드와 나란히 놓고 다른 줄을 찾아보세요. 몇 줄이 다른가요?

`model.config.pad_token_id` 를 설정하는 줄을 지우면 어떤 일이 생기나요?

21-03 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 GPT-2의 입력 맨 앞에는 BERT처럼 [CLS] 토큰이 붙는다. (O / X) 이름 _____

2 GPT의 첫 토큰 자리의 출력 벡터는 문장 전체의 정보를 담고 있다. (O / X) 이름 _____

3 GPT로 문장을 분류할 때 선형층에 넣는 것은? 이름 _____

① 첫 토큰의 벡터 ② 마지막 실제 토큰의 벡터 ③ 패딩 토큰의 벡터

4 이 실습의 출력층은? 이름 _____

① 768 → 2 ② 768 → 3 ③ 256 → 3

5 배치 안에서 길이를 맞추려고 채우는 토큰을 _____ 토큰이라고 한다. 이름 _____

6 실습에서 1 에포크 뒤의 테스트 정확도는 _____였다. 이름 _____

7 BERT는 맨 앞의 벡터로, GPT는 맨 뒤의 벡터로 분류하는 이유를 "보는 방향"으로 설명해 보세요. 이름 _____

원문: 21-03 GPT-2를 이용한 한국어 뉴스 긍정, 부정 감성 분류 — https://wikidocs.net/217619 (교재는 전체 데이터를 GPU에서 여러 에포크 학습시키고 검증 데이터로 가장 좋은 모델을 고릅니다. 여기서는 CPU에서 돌리려고 학습 1,200문장, 테스트 300문장, 1 에포크로 줄였고 검증 데이터 분리는 생략했습니다. 토크나이저는 21-02와 같은 이유로 PreTrainedTokenizerFast를 썼습니다)

21-04 GPT-2를 이용한 챗봇 파인 튜닝

핵심 한 줄

질문과 답을 특별 토큰으로 구분해 한 줄의 글로 이어 붙이고 평소처럼 "다음 토큰 맞히기"로 학습시키면, GPT는 질문 뒤에 답을 이어 쓰는 챗봇이 된다.

왜 배우나

20-06의 챗봇은 있는 답 중에서 골랐습니다. 이번에는 답을 직접 써내는 챗봇입니다. 출력층을 새로 붙이지 않고 **입력의 모양만 바꿔서** 파인 튜닝한다는 점이 지금까지와 다릅니다. ChatGPT 같은 대화형 모델도 같은 원리로, 대화를 한 줄의 글로 만들어 이어 쓰게 합니다.

1) 대화를 한 줄의 글로 만들기

정의

- 생성 기반 챗봇(generative chatbot): 답을 한 토큰씩 새로 써내는 챗봇. 20-06의 검색 기반 챗봇과 대비됩니다.
- 특별 토큰으로 역할 구분하기: KoGPT-2의 단어 집합에는 `<usr>` (사용자), `<sys>` (시스템), `</s>` (글의 시작과 끝) 같은 토큰이 들어 있습니다. 이것으로 누가 한 말인지와 어디서 끝나는지를 표시합니다.

$$\text{학습 글} = \texttt{</s>}\ \texttt{<usr>}\ \text{질문}\ \texttt{<sys>}\ \text{답}\ \texttt{</s>}$$

$$\text{손실} = -\frac{1}{T}\sum_{t=1}^{T}\log P(w_{t+1} \mid w_1, \ldots, w_t)$$

기호	뜻
$w_1, \ldots, w_{T+1}$	학습 글의 토큰들
$P(w_{t+1} \mid \cdot)$	t번째 자리까지 읽고 실제 다음 토큰에 준 확률

사전 훈련 때와 똑같은 손실입니다. 정답은 따로 없고, 입력을 한 칸 민 것이 정답입니다. 허깅페이스에서는 `labels` 에 입력과 같은 것을 넣으면 알아서 한 칸 밀어 계산합니다. 패딩 자리는 -100으로 채워 손실에서 뺍니다(20-03).

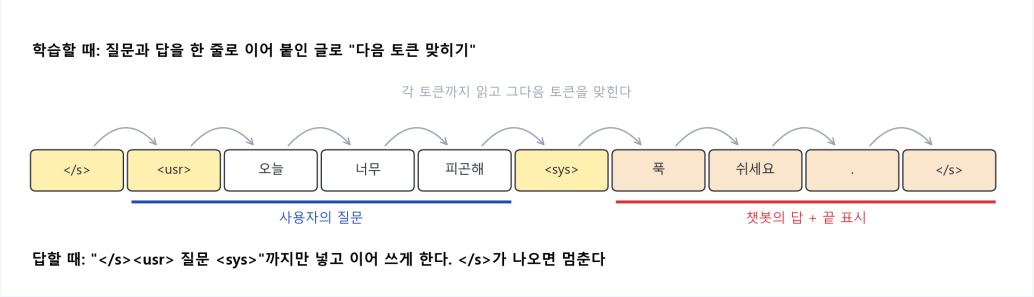


그림 5 학습할 때의 글과 답할 때의 입력

답할 때. `</s><usr>` 질문 `<sys>` 까지만 넣고 이어 쓰게 합니다. 학습 글에서 `<sys>` 뒤에는 늘 답이 왔으므로 모델은 답을 씁니다. 답 뒤에는 늘 `</s>` 가 왔으므로, `</s>` 가 나오면 생성을 멈춥니다(`eos_token_id`).

예시 적용

학습 글의 예. `</s><usr> 소오름 쫙<sys> 좋은 일이길 바랍니다.</s>` — 질문-답 2,000쌍을 이렇게 만들어 1 에포크 학습시켰습니다(실습 4).

나	파인 튜닝 전	파인 튜닝 후
오늘 너무 피곤해	이제부터 시작이오. 오늘은 내일도 쉬어야겠소. 이번에 새로 나온 신제품…	괜찮아요.
여행 가고 싶다	이제부터 나는 여행을 떠나기 전에, 내가 여행한 곳을 돌아보고…	괜찮아요.
배고파	배고픔을 참지 못하는 자는 그 고통을 참아내야 한다…	괜찮아요.
넌 누구니	넌 누구를 위한 것이냐?" "그렇습니다. 저는…	썸이랑 연락하는게 좋을 거예요.

- **배운 것:** 형식입니다. 파인 튜닝 전에는 소설이나 기사처럼 길게 이어 쓰다가 잘렸습니다. 사전 훈련 때 본 글이 그런 글이기 때문입니다. 파인 튜닝 뒤에는 짧은 한 문장을 쓰고 `</s>`에서 멈춥니다.
- **아직 못 배운 것:** 내용입니다. 질문과 상관없이 거의 모두 "괜찮아요."라고 답합니다. 위로하는 답이 많은 데이터에서, 아무 질문에나 무난한 답 하나로 손실을 줄이는 쪽으로 먼저 배운 것으로 보입니다. 가장 확률이 높은 토큰만 고르는 방식(21-02의 그리디)이라 같은 답이 그대로 나옵니다.
- 교재처럼 11,823쌍 전체로 여러 에포크 학습시키면 질문에 맞는 답이 나오기 시작합니다. 직접 해 볼 때는 `N_TRAIN`과 `EPOCHS`를 늘리고, `generate`에 `do_sample=True, top_k=10`을 넣어 보세요.

비유

대본을 많이 읽은 배우

"손님: … / 점원: …" 형식의 대본을 수천 장 읽은 배우에게 "손님: 배고파 / 점원:"까지만 보여 주면, 그 뒤에 점원의 대사를 이어서 말합니다. 배우는 여전히 "다음에 올 말"을 잇고 있을 뿐입니다.

손님: / 점원: = = <usr> / <sys>
대본 수천 장 읽기 = = 대화 글로 파인 튜닝
"점원:"까지만 보여 주기 = = 답할 때의 입력
대사가 끝났다는 표시 = = </s>

실습 4

KoGPT-2를 챗봇 데이터로 파인 튜닝하기

```
# GPT-2를 챗봇 데이터로 파인 튜닝한다 (CPU로 약 15분)
import urllib.request

import pandas as pd
import torch
from torch.utils.data import DataLoader
from transformers import GPT2LMHeadModel, PreTrainedTokenizerFast

N_TRAIN, EPOCHS = 2000, 1                              # 교재는 11,823쌍 전체, GPU
torch.manual_seed(0)
tokenizer = PreTrainedTokenizerFast.from_pretrained(
    "skt/kogpt2-base-v2", bos_token="</s>", eos_token="</s>", unk_token="<unk>",
```

```
    pad_token="<pad>", mask_token="<mask>")
model = GPT2LMHeadModel.from_pretrained("skt/kogpt2-base-v2")
print("특별 토큰:", {t: tokenizer.convert_tokens_to_ids(t)
                  for t in ["</s>", "<usr>", "<pad>", "<sys>"]})

# 1) 데이터: 질문과 답을 한 줄의 글로 이어 붙인다
url = "https://raw.githubusercontent.com/songys/Chatbot_data/master/ChatbotData.csv"
urllib.request.urlretrieve(url, "ChatBotData.csv")
data = pd.read_csv("ChatBotData.csv").sample(N_TRAIN, random_state=0)

def to_text(question, answer=""):
    return "</s><usr> " + question + "<sys> " + answer   # 답 뒤에는 </s>를 붙인다

samples = [tokenizer.encode(to_text(q, a) + "</s>")
           for q, a in zip(data["Q"], data["A"])]
print("학습 글의 예:", tokenizer.decode(samples[0]))

def collate(batch):                                      # 배치 안에서 길이를 맞춘다
    length = max(len(x) for x in batch)
    ids = torch.full((len(batch), length), tokenizer.pad_token_id)
    for i, x in enumerate(batch):
        ids[i, :len(x)] = torch.tensor(x)
    return ids

loader = DataLoader(samples, batch_size=16, shuffle=True, collate_fn=collate)

# 2) 답하기: "<usr> 질문 <sys>"까지 주고 그 뒤를 이어 쓰게 한다
def chat(question):
    ids = tokenizer.encode(to_text(question), return_tensors="pt")
    model.eval()
    with torch.no_grad():
        out = model.generate(ids, max_new_tokens=24, repetition_penalty=1.3,
                             eos_token_id=tokenizer.eos_token_id,
                             pad_token_id=tokenizer.pad_token_id)
    return tokenizer.decode(out[0, ids.size(1):], skip_special_tokens=True).strip()

questions = ["오늘 너무 피곤해", "여행 가고 싶다", "배고파", "넌 누구니"]
print("\n[파인 튜닝 전]")
for q in questions:
    print(f"  나: {q} → 챗봇: {chat(q)[:60]}")

# 3) 파인 튜닝: 평소처럼 "다음 토큰 맞히기"로 학습한다. 정답은 입력 그 자체다
optimizer = torch.optim.Adam(model.parameters(), lr=3e-5)
for epoch in range(1, EPOCHS + 1):
    model.train()
    total = 0.0
    for ids in loader:
        pad = ids == tokenizer.pad_token_id
        labels = ids.masked_fill(pad, -100)              # 패딩 자리는 손실에서 뺀다
        loss = model(input_ids=ids, attention_mask=~pad, labels=labels).loss
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total += loss.item()
    print(f"\n에포크 {epoch}: 평균 손실 {total / len(loader):.4f}")

print("[파인 튜닝 후]")
for q in questions:
    print(f"  나: {q} → 챗봇: {chat(q)[:60]}")
```

실행 결과

```
특별 토큰: {'</s>': 1, '<usr>': 2, '<pad>': 3, '<sys>': 4}
학습 글의 예: </s><usr> 소오름 쫙<sys> 좋은 일이길 바랍니다.</s>

[파인 튜닝 전]
  나: 오늘 너무 피곤해 → 챗봇: 이제부터 시작이오.
오늘은 내일도 쉬어야겠소. 이번에 새로 나온 신제품
  나: 여행 가고 싶다 → 챗봇: |
이제부터 나는 여행을 떠나기 전에, 내가 여행한 곳을 돌아보고 싶은 마음이 간절하다.
그곳은
  나: 배고파 → 챗봇: |
배고픔을 참지 못하는 자는 그 고통을 참아내야 한다.
그런데 이 고통은 다른 사람에게
```

```
  나: 넌 누구니 → 챗봇: 넌 누구를 위한 것이냐?"
"그렇습니다. 저는 그분께서도 아시다시피 이번에 제가 직접

에포크 1: 평균 손실 3.6134
[파인 튜닝 후]
  나: 오늘 너무 피곤해 → 챗봇: 괜찮아요.
  나: 여행 가고 싶다 → 챗봇: 괜찮아요.
  나: 배고파 → 챗봇: 괜찮아요.
  나: 넌 누구니 → 챗봇: 썸이랑 연락하는게 좋을 거예요.
```

직접 해 보기 질문을 바꿔 보세요. 학습 데이터에 없을 법한 질문에는 어떻게 답하나요? `chat`의 `generate`에 `do_sample=True, top_k=10`을 넣으면 같은 질문에 답이 어떻게 달라지나요?

확인 문제

21-04 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 이 챗봇을 만들려고 GPT 위에 새 출력층을 붙였다. (O / X) 이름 _____

2 챗봇 파인 튜닝의 손실은 사전 훈련 때와 같은 "다음 토큰 맞히기"의 손실이다. (O / X) 이름 _____

3 답할 때 모델에 넣는 입력은? 이름 _____

① 질문만 ② </s><usr> 질문 <sys> ③ </s><usr> 질문 <sys> 답 </s>

4 생성을 멈추는 신호가 되는 토큰은? 이름 _____

① <usr> ② <sys> ③ </s>

5 패딩 자리의 정답을 _____으로 채우면 손실에서 빠진다. 이름 _____

6 20-06의 챗봇은 검색 기반, 이 절의 챗봇은 _____ 기반이다. 이름 _____

7 검색 기반 챗봇(20-06)과 생성 기반 챗봇(21-04)의 장단점을 하나씩 말해 보세요. 이름 _____

원문: 21-04 GPT-2를 이용한 챗봇 파인튜닝 — https://wikidocs.net/217620 (교재는 11,823쌍 전체를 GPU에서 학습시킵니다. 여기서는 CPU에서 돌리려고 2,000쌍, 1 에포크로 줄였습니다. 파인 튜닝 전의 답과 비교하는 부분은 추가한 것입니다)

오늘의 요약

다시 보기



정의 모음

개념	정의	예시
GPT	트랜스포머 디코더만 쌓은 언어 모델. 앞만 본다	KoGPT-2: 12층, 1억 2,516만 개
입력과 출력	자리마다 다음 토큰의 점수	(1, 3) → (1, 3, 51200)
자기 회귀 생성	고른 토큰을 입력 뒤에 붙여 다시 넣기	근육이 커지기 위해서는 → 무엇보다 → …
디코딩 방법	확률 분포에서 토큰을 고르는 규칙	그리디, top-k, 반복 벌점
GPT로 분류	마지막 실제 토큰의 벡터 → 선형층	768 → 3
GPT로 챗봇	질문과 답을 한 줄로 이어 다음 토큰 맞히기로 학습	</s><usr> 질문 <sys> 답 </s>

20장과 나란히

	BERT (20장)	GPT (21장)
분류에 쓰는 벡터	맨 앞 [CLS]	마지막 실제 토큰
챗봇	문장 벡터로 비슷한 질문을 찾아 그 답을 돌려준다 (20-06)	답을 한 토큰씩 써낸다 (21-04)
파인 튜닝 때 바꾸는 것	출력층을 새로 붙인다	분류는 출력층을, 생성은 입력 글의 모양만 바꾼다

다음

다음 자료: 24장 랭체인으로 만드는 RAG (10/16 자료 4/5)

GPT는 그럴듯한 다음 토큰을 고를 뿐이라, 모르는 것도 그럴듯하게 씁니다. 다음 자료에서는 질문과 관련된 문서를 먼저 찾아 프롬프트에 넣어 주고, 그것을 보고 답하게 하는 RAG를 만듭니다.

정답과 해설

21-01 지피티(GPT)

1. X — 디코더를 쌓았습니다. 인코더를 쌓은 것은 BERT입니다.
2. X — 구조는 거의 그대로이고 크기와 데이터가 커졌습니다.
3. ② — 읽어 올 인코더가 없습니다.
4. ③
5. 거대 언어 모델
6. 51200
7. 예시 답: 마스크드 셀프 어텐션이 각 자리에서 뒤쪽 토큰을 가리기 때문이다. 앞 자리의 출력은 그 자리와 그 앞의 토큰으로만 계산되므로, 뒤의 토큰을 바꿔도 달라지지 않는다.

21-02 GPT-2를 이용한 문장 생성

1. X — 한 토큰씩 고르고 붙여 다시 넣기를 반복합니다.
2. X — 늘 가장 높은 것을 고르므로 같은 글이 나옵니다.
3. ②
4. ①
5. 자기 회귀
6. 0.02
7. 예시 답: 첫 토큰으로 4등이던 "피부"가 뽑혔고, 그 토큰이 입력에 붙어 다음 토큰의 확률을 모두 바꿨다. 모델은 지금까지 쓴 글에 이어질 그럴듯한 말을 고를 뿐이어서, 한 번 피부 이야기가 시작되면 계속 그쪽으로 이어진다.

21-03 GPT-2를 이용한 한국어 뉴스 감성 분류

1. X — GPT-2에는 [CLS]가 없습니다.
2. X — 첫 토큰의 자리는 첫 토큰만 봤습니다.
3. ②
4. ②
5. 패딩
6. 0.8033
7. 예시 답: BERT는 양쪽을 모두 보므로 맨 앞의 [CLS] 자리에서도 문장 전체를 참고할 수 있다. GPT는 앞만 보므로 문장 전체를 본 자리는 마지막 토큰뿐이다.

21-04 GPT-2를 이용한 챗봇 파인 튜닝

1. X — 출력층은 그대로이고 입력 글의 모양만 바꿨습니다.
2. O
3. ②
4. ③
5. -100
6. 생성
7. 예시 답: 검색 기반은 준비된 답만 내므로 말이 이상해질 일이 없지만, 준비되지 않은 질문에는 엉뚱한 답을 고른다. 생성 기반은 처음 보는 질문에도 답을 써내지만, 어색하거나 틀린 말을 지어낼 수 있다.

용어 정리

용어	정의
GPT	트랜스포머 디코더를 쌓고 다음 단어 맞히기로 사전 훈련한 언어 모델
거대 언어 모델 (LLM)	파라미터 수가 충분히 큰 딥 러닝 언어 모델
디코더 전용 구조	트랜스포머 디코더 층에서 인코더-디코더 어텐션을 뺀 구조
마스크드 셀프 어텐션	뒤쪽 토큰을 가려 앞의 토큰만 참고하게 한 셀프 어텐션
자기 회귀 생성	모델의 출력을 다음 입력에 붙여 다시 넣는 방식의 생성
그리디 / top-k 샘플링	확률이 가장 높은 토큰을 고르기 / 상위 k개 중에서 뽑기
반복 벌점	이미 쓴 토큰의 점수를 깎아 되풀이를 줄이는 것
KoGPT-2	SKT가 한국어 글로 사전 훈련해 공개한 GPT-2
<usr> / <sys> / </s>	사용자의 말 / 시스템의 말 / 글의 시작과 끝을 나타내는 특별 토큰
생성 기반 챗봇	답을 한 토큰씩 새로 써내는 챗봇